

การแบ่งหน้าที่ทีมพัฒนา

BidNest — Auction & Marketplace

ทีมพัฒนา 5 คน · อัปเดตให้ตรงกับ SRS ฉบับล่าสุด

เอกสารนี้แบ่งงานทีม 5 คนโดยอ้างอิงโครงสร้างจากตัวอย่างที่ทีมกำหนดไว้ในตอนแรก แล้วปรับให้สอดคล้องกับ SRS ฉบับล่าสุด (v1.0) ทั้งหมด เพื่อให้ไม่ให้ scope งานของแต่ละคนหลุดจากข้อกำหนดที่ตกลงกันได้

ภาพรวม

Dev	บทบาท	ความรับผิดชอบหลัก	Requirement
Dev 1	Frontend Lead & Design System	Design system, shared UI, การแจ้งเตือน/โปรไฟล์/Watchlist, NextAuth ผัง client, protected routes	NOT-001..007 (+ NOT-008 Optional ตาม CHAT), USR-001, AUTH-008
Dev 2	Backend Core & Security	NestJS setup, Prisma schema หลัก, ระบบยืนยันตัวตนทั้งหมด, ความปลอดภัยโดยรวม	AUTH-001..008, USR-001, ADM-003, §6
Dev 3	E-Commerce Module	แคตตาล็อกสินค้า, ส่วนลดอัตโนมัติตามจำนวน, ตะกร้า+checkout จำลอง, ติดตามจัดส่ง+ประวัติคำสั่งซื้อ	PROD-001..007, CART-001..005, SHIP-001..003, ADM-005/006, CHAT-001..003(Optional)
Dev 4	Auction Module & Real-time	วงจรวีดิประมูล, การประมูลเรียลไทม์, Live Arena, การแจ้งเตือนฝั่งประมูล	AUC-001..008, BID-001..005, WAT, NOT-001..004, LIV-001..005, ADM-001
Dev 5	AI Specialist & Admin Dashboard	Customer Service AI Chatbot (บังคับ), Admin Dashboard รวมศูนย์ · AI Price Estimator/Negotiator (Optional)	AI-001, ADM-002, ADM-004 (+ AI-002, AI-003 Optional)

รายละเอียดแต่ละตำแหน่ง

Dev 1 — Frontend Lead & Design System

รับผิดชอบโครงสร้าง Next.js App Router, Design System ด้วย Shadcn-UI, Responsive Layout และ shared component ที่ทุกโมดูลใช้ร่วมกัน ได้แก่ ระบบแจ้งเตือนในแอป, หน้าโปรไฟล์, Watchlist UI, การเชื่อมต่อ NextAuth ผัง client และ protected-route middleware

Requirement ที่เกี่ยวข้อง: NOT-001..007 (+ NOT-008 Optional ตาม CHAT) , USR-001, AUTH-008

Dev 2 — Backend Core & Security

รับผิดชอบ NestJS setup, ออกแบบ Prisma schema หลักและดูแล migration ให้ทั้งหมด, ระบบยืนยันตัวตนทั้งหมด (Local login, Google/Line OAuth, 2FA แบบ OTP ทางอีเมลที่บังคับทุกบัญชี, NextAuth ผัง server, Maildev/SMTP relay, JWT Access/Refresh + NestJS guard), การจัดการหมวดหมู่รวม และข้อกำหนดความปลอดภัยโดยรวมของระบบ

Requirement ที่เกี่ยวข้อง: AUTH-001..008, USR-001, ADM-003, §6 (Security and quality requirements)

Dev 3 — E-Commerce Module

รับผิดชอบ REST API และหน้าจอฟัง e-commerce ทั้งหมด ได้แก่ แคตตาล็อกสินค้า (รวม negotiation floor และส่วนลดอัตโนมัติตามจำนวนซื้อ), ตะกร้าสินค้าและ checkout แบบจำลอง, การติดตามจัดส่งและประวัติคำสั่งซื้อ, และฟังก์ชัน Admin เฉพาะโมดูลของตัวเอง

Requirement ที่เกี่ยวข้อง: PROD-001..007, CART-001..005, SHIP-001..003, ADM-005, ADM-006, CHAT-001..003(Optional)

Dev 4 — Auction Module & Real-time Specialist

รับผิดชอบ REST API และหน้าจอฝั่งประมูลทั้งหมด ได้แก่ วงจรชีวิตของประมูล, การประมูลแบบเรียลไทม์ผ่าน WebSocket Gateway, ระบบ Anti-sniping, Live Arena, Watchlist ฝั่งประมูล, การแจ้งเตือนฝั่งประมูล และฟังก์ชัน Admin เฉพาะโมดูลของตัวเอง

Requirement ที่เกี่ยวข้อง: AUC-001..008, BID-001..005, WAT-001/002, NOT-001..004, LIV-001..005, ADM-001

Dev 5 — AI Specialist & Admin Dashboard

รับผิดชอบ Customer Service AI Chatbot ที่ช่วยตอบปัญหาการใช้งานเบื้องต้นให้ผู้ใช้ (AI-001) ซึ่งเป็นพีเจอาร์ AI ที่บังคับต้องทำ รวมถึง Admin Dashboard แบบรวมศูนย์ที่เรียกใช้ endpoint การจัดการผู้ใช้และ audit log ซึ่งอิงจากโมดูลของ Dev 2/3/4 หากมีเวลาเหลือหลังจากงานหลักเสร็จสมบูรณ์แล้ว สามารถทำพีเจอาร์เสริม (Optional) เพิ่มเติมได้ คือ AI Price Estimator สำหรับผู้ขายฝั่งประมูล (AI-002) และ AI Negotiator สำหรับผู้ซื้อฝั่ง e-commerce (AI-003)

Requirement ที่เกี่ยวข้อง: AI-001, ADM-002, ADM-004 (บังคับ) · AI-002, AI-003 (Optional)

ขั้นตอนเริ่มต้นโปรเจก (Git → Docker → Jira)

ลำดับนี้ออกแบบให้ Dev 2 (auth) ไม่ติด block รอสิ่งอื่นตั้งแต่ต้น ส่วน Jira (ขั้นตอนที่ 5) ทำคู่ขนานไปกับขั้นตอนที่ 2-4 ได้เลย ไม่จำเป็นต้องรอเรียงตามลำดับเป๊ะ

- ตั้งค่า Git Repository** — สร้าง repo บน GitHub/GitLab (private) เพิ่ม README.md และ .gitignore สำหรับ Node/Next.js/NestJS ตั้ง branch strategy: main (protected, merge ผ่าน PR เท่านั้น อย่างน้อย 1 approve) + dev branch + feature branch แยกตามคน เช่น feat/auth-dev2, feat/ecommerce-dev3, feat/auction-dev4, feat/ai-dev5, feat/frontend-dev1
- Scaffold โครงสร้าง Monorepo** — ใช้ pnpm workspace แบ่งเป็น apps/web (สร้างด้วย create-next-app, App Router), apps/api (สร้างด้วย nest new), packages/contracts (เก็บ shared type/DTO ระหว่าง frontend-backend), packages/config (shared ESLint/TypeScript config) สร้าง pnpm-workspace.yaml ที่ root แล้ว commit โครงสร้างเป่านั้นเป็น commit แรกเข้า dev branch
- ตั้งค่า Docker Compose (Postgres + Maildev)** — สร้าง infra/docker/compose.dev.yml มี 2 services คือ postgres (image postgres:17, map port เป็น 5433 กันชนกับ Postgres ที่อาจมีอยู่ในเครื่องแล้ว) และ maildev สร้าง .env.example ที่ root ระบุ DATABASE_URL ขึ้นที่ container รันทดสอบด้วย docker compose -f infra/docker/compose.dev.yml up -d แล้วเช็คว่ายี่ห้อ Postgres ได้และหน้า Maildev (port 1080) เข้าถึงได้
- ตั้งค่า Prisma และ Schema เริ่มต้น** — ติดตั้ง Prisma ใน apps/api เชื่อมกับ Postgres ใน Docker ผ่าน DATABASE_URL สร้าง schema เริ่มต้นเฉพาะส่วนที่ทุกคนรอใช้ก่อน คือ Identity (users, user_profiles, auth_accounts, user_sessions, two_factor_codes) และ categories ที่ใช้ร่วมกันสองโมดูล รัน migration แรกด้วย prisma migrate dev แล้ว commit schema.prisma พร้อมไฟล์ migration เข้า dev branch ให้ทุกคน pull ไปใช้ได้
- สร้าง Jira Project และ Backlog ตาม SRS** — สร้าง Jira project ตั้ง Epic ตามหมวดใน SRS 5 Epic คือ Authentication, Auction, E-commerce, AI Features, Admin แยกแต่ละ Epic เป็น Story ตาม requirement ID ตรงๆ เช่น AUTH-001 ถึง AUTH-008 เป็น 8 story ใน Epic Authentication ส่วน Epic AI Features ให้ตั้ง story ของ AI-001 (Customer Service Chatbot) เป็น priority บังคับ และตั้ง story ของ AI-002/AI-003 เป็น priority ต่ำสุดแบบ Optional/stretch แยกไว้ชัดเจน มอบหมาย assignee ตามตารางภาพรวมด้านบน และตั้ง Sprint แรกให้ story ของ Dev 2 (auth พื้นฐาน) เป็น priority สูงสุด
- ตั้งค่า CI พื้นฐาน (Lint + Test บน PR)** — เพิ่ม GitHub Actions workflow (.github/workflows/ci.yml) ให้รัน lint และ test อัตโนมัติทุกครั้งที่เปิด PR เข้า main หรือ dev ผูกกับ branch protection ให้ CI ต้องผ่านก่อน merge ได้จริง เพื่อรองรับข้อกำหนด automated coverage ตาม SRS §6
- Kickoff ทีมและเริ่มงานคู่ขนาน** — Dev 2 เริ่ม auth พื้นฐาน (local login + JWT access/refresh) จาก story priority สูงสุดทันที Dev 1 เริ่ม design system คู่ขนานได้เลยเพราะไม่ต้องรอ auth เสร็จก่อน ส่วน Dev 3, Dev 4, Dev 5 เริ่ม scaffold โครงสร้างโมดูลของตัวเอง (routes, DTO) โดย mock auth ไปก่อน แล้วค่อยเชื่อมต่อจริงเมื่อ auth พื้นฐานของ Dev 2 พร้อมใช้งาน